

G209 — Devolución sobre investigación y pruebas técnicas de Matricular

Hola Iván y Nicolás.

Estuve revisando con bastante atención el material adicional y los scripts que desarrollaron. Considero que **hicieron un muy buen trabajo y respondieron directamente a lo solicitado en la devolución anterior.**

Valoro especialmente que hayan investigado fuentes reales, analizado técnicamente cómo funcionan, realizado una prueba concreta con Ecogas y respetado correctamente el límite encontrado en MetroGAS sin intentar sortear el reCAPTCHA.

También considero acertada la decisión de reducir el primer corte vertical a **Ecogas**, que es la fuente sobre la cual pudieron demostrar viabilidad técnica.

Con los archivos Python incluidos, además, queda claro que **la prueba técnica mínima que habíamos solicitado ya está cumplida.**

En prueba_ecogas.py no se limitaron a verificar conectividad: recorrieron una primera porción vertical del problema:

fuentes reales → descarga → extracción → estructuración → aplicación de una regla → resultado.

Por lo tanto, ya no considero pendiente demostrar que pueden obtener y procesar técnicamente información de Ecogas.

Del mismo modo, la prueba realizada contra MetroGAS me parece correcta: verificaron mediante código que la fuente exige un captcha válido y deliberadamente no intentaron sortear esa protección. Eso documenta un límite técnico real sin intentar vulnerarlo. Muy bien.

Pero la investigación produjo algo todavía más valioso: **permitió descubrir que algunas de las reglas que estaban empezando a automatizar no eran correctas.**

Y justamente para eso sirve esta etapa.

Jurisdicción y alcance de la matrícula

El primer punto que deben corregir es la interpretación del campo provincia.

En la prueba toman un gasista registrado en Córdoba y evalúan un trabajo ubicado en San Luis. El código concluye que el profesional no corresponde porque la provincia no coincide.

Esa conclusión no está suficientemente respaldada por la regla del dominio que relevamos.

La documentación que analizaron indica que determinadas categorías de matrícula poseen un alcance territorial más amplio que la provincia que figura en el padrón. Por lo tanto, **el campo provincia no puede interpretarse automáticamente como “provincia donde este profesional está habilitado para trabajar”**.

Necesitamos diferenciar:

lugar/distribuidora donde aparece registrado el matriculado

de

alcance efectivo de la habilitación profesional.

Por lo tanto, retiren del modelo y del código, por ahora, una regla del tipo:

provincia del padrón != provincia del trabajo → no habilitado

hasta tener una regla formal que permita determinar realmente esa incompatibilidad.

Éste es un resultado muy importante del trabajo realizado: **el programa funcionó correctamente según la regla que ustedes escribieron, pero produjo una conclusión incorrecta porque la regla de negocio utilizada no era correcta.**

Eso demuestra algo fundamental para el Proyecto Final:

Antes de automatizar una decisión debemos validar la regla que estamos automatizando.

Categorías: tampoco son una comparación numérica

Encontré el mismo problema en la evaluación de categorías.

Actualmente el script utiliza conceptualmente:

`categoria_obtenida >= categoria_requerida`

Pero las categorías profesionales no deben modelarse simplemente como un ranking numérico donde un número mayor implica mayor capacidad.

Las categorías representan **alcances técnicos diferentes**.

Por lo tanto, no deberían decidir la elegibilidad mediante una comparación numérica.

El sistema debería poder conocer qué habilita cada categoría y luego comparar esas capacidades contra las características del trabajo solicitado.

Para P0 no necesitan construir un motor regulatorio universal. Seleccionen uno o dos casos de trabajo suficientemente claros, documenten qué categoría los puede realizar y construyan las reglas sobre esa evidencia.

Esto es mucho más sólido que trabajar con:

1, 2, 3

como si fueran niveles crecientes.

Credencial no es lo mismo que elegibilidad

A partir de estos hallazgos recomiendo mantener una separación conceptual fuerte en el modelo.

Credencial debería representar:

qué matrícula encontramos, en qué fuente, qué categoría informa, cuándo se realizó la consulta y qué resultado produjo esa consulta.

En cambio, la **evaluación de elegibilidad** debería responder:

si la información disponible permite determinar que esa credencial resulta adecuada para un trabajo concreto y por qué.

Esto significa que estados como `categoria_insuficiente` —y eventualmente una incompatibilidad territorial si después logran demostrarla— **no deberían ser estados permanentes de la credencial**.

La misma credencial puede resultar suficiente para un trabajo y no para otro.

Conceptualmente:

Credencial → evidencia sobre el profesional.

Evaluación → resultado de aplicar reglas de un trabajo sobre esa credencial.

Esa distinción les va a simplificar mucho el modelo.

Vigencia: distinguir dato individual de regla general

La investigación también muestra que Ecogas no incorpora una fecha de vencimiento individual dentro de cada registro obtenido.

Eso no significa necesariamente que no exista una regla general sobre renovación o vigencia.

Por eso ahora aparece una pregunta mucho más interesante:

¿La presencia de una persona en el padrón público significa que su matrícula está actualmente renovada y habilitada?

Ésa es la relación que todavía necesitan verificar.

Si pueden consultar directamente a Ecogas sobre el criterio de inclusión y la frecuencia con que actualizan el padrón, sería muy útil.

Mientras esa relación no esté confirmada, evitaría incorporar al modelo operativo una fecha_vencimiento ficticia solamente para completar estados posibles.

Los datos ficticios son válidos para pruebas, pero el sistema P0 debe distinguir claramente entre:

lo que realmente puede conocer

y

lo que solamente puede simular.

“No encontrado” y “no verificable” son resultados diferentes

Mantengan especialmente esta distinción porque está muy bien encaminada.

No es lo mismo:

Ecogas fue consultado correctamente y una matrícula no apareció

que:

MetroGAS no pudo ser consultado automáticamente porque exige captcha.

En el primer caso existe un resultado negativo respecto de una fuente concreta.

En el segundo existe **incertidumbre**.

Por eso tampoco utilizaría expresiones generales como:

“no aparece en ningún padrón”

si algunas fuentes relevantes no pudieron consultarse.

El sistema debería poder decir, por ejemplo:

Fuente Ecogas: consulta OK → matrícula no encontrada.

Fuente MetroGAS: consulta no realizada automáticamente → resultado desconocido.

La ausencia de evidencia no debe convertirse en una certeza.

Ésta puede ser una invariante importante de Matricular:

Si el sistema no dispone de evidencia suficiente para afirmar algo, debe expresar la incertidumbre y no completar la respuesta por inferencia.

Acceso a Ecogas y estabilidad de la fuente

La prueba demuestra que técnicamente pudieron acceder al recurso público de Ecogas sin autenticación ni captcha.

Cambiaría, sin embargo, la expresión **“automatizable de forma legítima”** por una formulación más prudente.

La prueba permite afirmar:

la fuente resultó técnicamente accesible en las condiciones ensayadas.

Eso no determina automáticamente cuáles son las condiciones permitidas para una explotación periódica o productiva de los datos.

Para el TPF pueden mantener esta integración como una **prueba técnica acotada**, documentando que se trata de una fuente web no contractual, cuya estructura y URL pueden cambiar.

Si en algún momento pretendieran convertir Matricular en un servicio operativo real, deberían revisar además las condiciones de utilización correspondientes.

Una precisión metodológica sobre el script

También hay un detalle pequeño, pero importante para que la documentación sea técnicamente exacta.

En el script indican que, por razones de minimización, guardan solamente una muestra de diez registros. Eso efectivamente ocurre al generar el JSON local.

Sin embargo, el programa **descarga y procesa el recurso completo** y después conserva sólo esa muestra.

Por lo tanto sería mejor documentar:

“Se procesa la fuente completa necesaria para realizar la prueba, pero únicamente se persiste una muestra reducida.”

Además, eviten incorporar en GitHub, capturas, informe o video nombres, teléfonos y correos reales que no sean necesarios para demostrar el funcionamiento.

Para las demostraciones pueden anonimizar esos campos y conservar solamente matrícula, categoría u otros datos estrictamente necesarios para el caso técnico.

Robustez del adaptador

El código actual utiliza una URL de chunk de Next.js fija y el extractor depende de la estructura actual del archivo.

Eso está perfectamente aceptable como **prueba de concepto**.

Ahora debe convertirse en una responsabilidad del adaptador.

Si Ecogas cambia la URL, el orden de campos o la estructura del recurso, Matricular no debe interpretar:

“se encontraron cero matriculados”.

Debe producir algo equivalente a:

FUENTE NO CONSULTABLE

o

EXTRACCIÓN FALLIDA.

Me parece especialmente correcta la idea que ustedes plantean de considerar una extracción inesperadamente vacía como posible error de la fuente.

Mantengan esa regla.

Lo que todavía no considero necesario es dar automáticamente por justificada una arquitectura compleja de colas y DLQ.

Primero implementen el mecanismo más sencillo que permita:

registrar el intento → detectar el error → reintentar → conservar el estado del fallo → informar.

Si después la cantidad de fuentes, procesos y verificaciones realmente requiere colas, ahí tendremos una justificación concreta para incorporarlas.

Siguiente paso: dejar los scripts aislados y comenzar Matricular

Con el avance actual considero que **la viabilidad técnica inicial está demostrada y pueden continuar con Arquitectura y Módulos.**

El próximo paso ya no debería ser crear otro script exploratorio en Python.

Ahora quiero que conviertan lo aprendido en el primer recorrido integrado del sistema:

Adaptador Ecogas → obtención/extracción → normalización de credencial → persistencia de fuente, fecha y resultado → evaluación mediante una regla de categoría correctamente documentada → respuesta del backend.

Si pueden agregar una interfaz mínima desde donde ejecutar o consultar ese proceso, tendremos ya un verdadero recorrido vertical de Matricular.

También quiero que implementen el camino de error:

fuentes modificadas/no disponibles → extracción fallida → resultado NO VERIFICABLE

sin borrar ni transformar incorrectamente la última evidencia válida disponible.

Para la próxima revisión me interesa especialmente ver:

1. modelo corregido separando **Credencial** y **Evaluación**;
2. eliminación de la comparación directa provincia=jurisdicción;
3. reglas reales de categoría documentadas;
4. adaptador Ecogas integrado al backend;
5. persistencia de fuente, fecha y resultado de consulta;
6. tratamiento explícito de NO_ENCONTRADO frente a NO_VERIFICABLE;
7. manejo controlado del fallo de extracción;
8. una interfaz mínima o endpoint que permita recorrer el flujo completo.

Con estos avances considero que **el proyecto está mucho mejor fundamentado que en su planteo inicial y puede continuar.**

Lo más valioso del trabajo realizado hasta ahora no es solamente haber demostrado que pueden extraer datos de una fuente real. Es que la investigación les permitió descubrir que **obtener correctamente un dato no implica necesariamente interpretarlo correctamente.**

Eso es exactamente el tipo de aprendizaje que buscamos durante Proyecto Final.

El desafío de Matricular no debería ser simplemente consultar padrones, sino poder responder con precisión:

qué sabemos, de dónde lo sabemos, cuándo lo verificamos, qué podemos concluir con esa evidencia y qué no podemos afirmar.

Si consiguen mantener esa disciplina durante el desarrollo, van a tener un núcleo técnicamente interesante y conceptualmente sólido.

Saludos,
Oscar

